# RPC TCP Application Variant 1

## Описание проекта

Прототип приложения с поддержкой удалённого вызова процедур (RPC) через TCP-соединение.

Приложение реализует взаимодействие клиента и сервера:

- клиент формирует XML-запрос;
- запрос передаётся через TCP-соединение;
- сервер принимает пакет;
- сервер обрабатывает запрос;
- сервер возвращает XML-ответ клиенту.

Данные хранятся только в оперативной памяти и не используются внешние базы данных.

---

## Используемые технологии

- Python 3.13
- TCP sockets
- XML
- pytest
- Hypothesis

---

## Структура проекта

```
rsc_pr1/

├── src/
│   ├── __init__.py
│   ├── database.py          # хранение данных в памяти
│   ├── models.py            # модели объектов
│   ├── protocol.py          # создание и чтение TCP пакетов
│   ├── rpc_client.py        # RPC клиент
│   ├── rpc_server.py        # RPC сервер
│   └── rpc_handler.py       # обработка RPC запросов
│
├── tests/
│   ├── __init__.py
│   ├── test_rpc.py          # модульные тесты
│   └── test_integration.py  # интеграционные тесты
│
├── README.md
└── .gitignore
```

---

# Установка и запуск

## Создание виртуального окружения

```
python -m venv venv
```

## Активация виртуального окружения

Windows:

```
venv\Scripts\activate
```

---

# Запуск сервера

Из корневой папки проекта выполнить:

```
python -m src.rpc_server
```

После успешного запуска появится сообщение:

```
RPC server started
```

Сервер ожидает подключения клиента.

---

# Запуск клиента

В отдельном терминале выполнить:

```
python -m src.rpc_client
```

Клиент подключается к серверу и отправляет RPC-запрос.

---

# Принцип работы RPC

В приложении используется собственный TCP-протокол обмена сообщениями.

Каждый пакет содержит:

- версию протокола;

- код операции;
- размер XML сообщения;
- данные XML.

Общий вид передачи:

```
Client
  |
  | TCP packet
  |
  v
Server
  |
  | XML processing
  |
  v
Response
```

---

# Поддерживаемые операции

## create_session

Создание новой пользовательской сессии.

Пример XML-запроса:

```
<request>
    <method>
        create_session
    </method>

    <uid>
        1
    </uid>

    <locale>
        ru
    </locale>
</request>
```

Ответ сервера:

```
<response>
    <status>success</status>

    <message>
        Session created
```

```
    </message>
</response>
```

---

## Обработка ошибок

Система поддерживает обработку некорректных XML-запросов.

Например, при повреждённом XML сервер не завершает работу с ошибкой, а возвращает:

```
<response>
    <status>error</status>

    <message>
        Invalid XML
    </message>
</response>
```

---

## Тестирование

Для запуска всех тестов используется:

```
pytest
```

В проекте реализованы проверки:

- создание RPC-сессии;
- обработка неправильного XML;
- создание и чтение TCP-пакетов;
- запуск RPC сервера.

Результат успешного выполнения:

```
4 passed
```

---

## Назначение компонентов

### models.py

Содержит классы объектов приложения:

- Session
- Prompt

- Feedback

---

## database.py

Отвечает за хранение объектов в оперативной памяти.

---

## protocol.py

Реализует упаковку и распаковку TCP сообщений.

Функции:

- создание пакета;
- чтение пакета.

---

## rpc_client.py

Клиентская часть приложения.

Отвечает за:

- подключение к серверу;
- отправку запроса;
- получение ответа.

---

## rpc_server.py

Серверная часть приложения.

Отвечает за:

- запуск TCP сервера;
- получение сообщений;
- передачу запроса обработчику;
- отправку ответа клиенту.

---

## rpc_handler.py

Обрабатывает поступившие RPC-команды и формирует ответы.

---